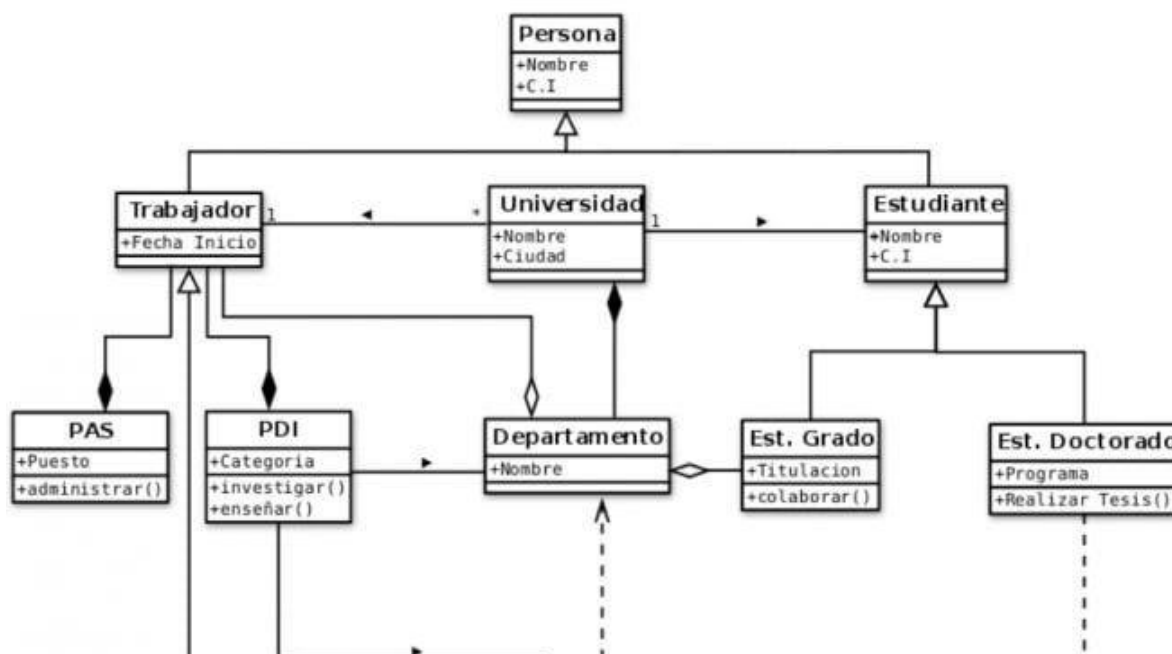




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Análisis y Diseño de Software I	Unidad 5	Tema: Identificación de clases del dominio
	Semana 13	Instructora: Katya Michelle Asencio Bernal

Introducción:

Bienvenido al segundo material de lectura de la Unidad #5 de Análisis y Diseño de Software I. En este módulo nos enfocaremos en la Identificación de clases del dominio y en cómo definir correctamente las responsabilidades de cada clase. Posteriormente, analizaremos los errores comunes que se deben evitar al modelar, acompañados de las buenas prácticas de diseño que garantizan la claridad del sistema. Finalmente, exploraremos el software y las herramientas modernas clave para la creación y mantenimiento de estos diagramas en el entorno profesional.

Este material de lectura busca brindarte una base sólida en campo del Análisis y Diseño de Software, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

Antes de comenzar, asegúrate de tener papel y lápiz a mano para las actividades interactivas.

Sección 1:

5.4 Identificación de clases del dominio

El dominio del problema define un modelo de clases común para todos los involucrados en el modelo de requisitos, tanto analistas como clientes. Este modelo de clases consiste en los objetos del dominio del problema, o sea objetos que tienen una correspondencia directa en el área de la aplicación. Como los usuarios y clientes deben reconocer todos los conceptos, se puede desarrollar una terminología común al razonar sobre los casos de uso y, por lo tanto, disminuir la probabilidad de malos entendidos entre el analista y el usuario.

Identificación de clases del dominio

La identificación de clases del dominio del problema, se obtiene principalmente de algún documento textual que describa el sistema. Aunque se pudiera tomar como punto de partida los documentos desarrollados para el modelo de casos de uso, a menudo la descripción original del problema es suficiente. Se comienza este proceso a partir de la identificación de las clases candidatas, explícitas o implícitas, a las que se refiera la descripción del problema. Para ello, se extraen todos los sustantivos de la descripción del problema o de algún otro documento similar, de acuerdo con las siguientes consideraciones:

- Los sustantivos en la descripción del problema son los posibles candidatos a clases de objetos.
- Se deben identificar las entidades físicas al igual que las conceptuales.
- No se debe tratar de diferenciar entre las clases y los atributos.
- Dado que no todas las clases se describen de manera explícita, siendo algunas implícitas en la aplicación, será necesario añadir clases que puedan ser identificadas por nuestro conocimiento del área.



Actividad Interactiva #1

Verdadero o Falso

___ Los sustantivos de la descripción del problema pueden representar posibles clases.

___ Los atributos siempre deben convertirse en clases independientes.

___ Las clases del dominio representan conceptos del área del problema.

___ El modelo de dominio ayuda a disminuir malos entendidos entre analistas y usuarios.

- Se deben revisar los pronombres en la descripción del problema, para asegurar que no se haya perdido ningún sustantivo descrito de forma implícita.
- Para facilitar la identificación de clases, se subrayan todos los sustantivos de la descripción del problema.

Proceso de Refinamiento (Depuración)

Una vez listadas las clases candidatas, se debe realizar un proceso de depuración para eliminar elementos innecesarios o incorrectos:

- **Eliminar Clases Redundantes:** Si dos nombres representan lo mismo (ej. *Cliente* y *Usuario*), se elige el más descriptivo para el contexto.
- **Eliminar Clases Irrelevantes:** Elementos que no son centrales para el sistema actual (ej. *Mostrador* si el sistema solo gestiona boletos en línea).
- **Eliminar Clases Imprecisas:** Términos vagos como *Sistema*, *Información* o *Estado* que no aportan valor concreto.
- **Convertir a Atributos:** Propiedades que pertenecen a una clase principal (ej. *Número de tarjeta* es un atributo de *Tarjeta de Crédito*, no una clase independiente).
- **Convertir a Operaciones:** Acciones descritas como sustantivos (ej. *Consulta*, *Pago*) deben moverse al comportamiento de la clase, no ser clases en sí mismas.

Representación

El resultado final se representa generalmente mediante un Diagrama de Clases Conceptuales en UML. Este diagrama muestra:

- Las clases identificadas (nombres comunes, singular).
- Las **asociaciones** entre ellas (relaciones de agregación, composición o generalización).
- La **multiplicidad** de las relaciones (1 a 1, 1 a n, n a n).
- Los **atributos** principales y roles de cada clase.

Este modelo no incluye interfaces gráficas, bases de datos ni detalles de implementación, enfocándose exclusivamente en el vocabulario del dominio para crear un entendimiento común entre analistas y expertos del área.

Sección 2:

5.4.1 Responsabilidades de las clases

Una vez identificadas las clases del dominio dentro del proceso de análisis y diseño de software, es necesario definir las responsabilidades que tendrá cada una de ellas.

Las responsabilidades de una clase representan el contrato u obligación que esta tiene dentro del sistema. Definen qué es lo que la clase *conoce* (su estado o atributos) y qué es lo que la clase *hace* (su comportamiento o métodos).

Al identificar las clases del dominio, no solo nos importa su nombre, sino qué rol juegan en la lógica del negocio. Una excelente manera de definir estas responsabilidades es dividir las en dos categorías principales:

- **Responsabilidades de Conocimiento (Saber):** Lo que la clase debe recordar o rastrear.
 - Datos sobre su propio estado (atributos).
 - Relaciones con otros objetos del sistema.
- **Responsabilidades de Comportamiento (Hacer):** Las acciones que la clase puede ejecutar.
 - Realizar cálculos u operaciones con sus propios datos.
 - Iniciar o coordinar acciones en otros objetos.
 - Crear o destruir instancias de otras clases si tiene el rol de creador.

¿Cómo identificar las responsabilidades?

Una de las técnicas más efectivas y dinámicas en esta etapa es el uso de **Tarjetas CRC (Clase-Responsabilidad-Colaboración)**. Para cada clase detectada en el dominio, nos hacemos tres preguntas clave:

1. **¿Quién soy?** (El nombre de la clase).
2. **¿Qué debo hacer o saber?** (Las responsabilidades).
3. **¿A quién necesito para lograrlo?** (Las colaboraciones con otras clases).

Una clase debe tener alta cohesión. Esto significa que todas sus responsabilidades deben estar estrechamente relacionadas con el concepto que la clase representa. Si una clase empieza a acumular responsabilidades que pertenecen a otros contextos, es una señal de que debe dividirse.

Sección 3:

5.4.2 Errores comunes en diagramas de clases

Los diagramas de clases son una herramienta fundamental en el análisis y diseño de software orientado a objetos, ya que permiten representar la estructura del sistema mediante clases, atributos, métodos y relaciones. Sin embargo, durante su elaboración es común cometer errores que afectan la comprensión, organización y funcionamiento del diseño.

1. Clases con demasiadas responsabilidades

Ocurre cuando una sola clase realiza muchas funciones diferentes dentro del sistema.

Problema:

- El código se vuelve difícil de mantener.
- Aumenta la complejidad del sistema.

Ejemplo: Una clase Sistema que administra usuarios, inventario, facturación y reportes al mismo tiempo.

Corrección:

Dividir las funciones en clases específicas como:

- Usuario
- Inventario
- Factura
- Reporte

2. Clases innecesarias o redundantes

Sucede cuando se crean clases que no aportan funcionalidad real al sistema.

Problema:

- El diseño se vuelve confuso.
- Se incrementa la cantidad de clases sin necesidad.

Ejemplo: Crear una clase ColorProducto cuando el color podría ser simplemente un atributo de Producto.

3. Relaciones incorrectas entre clases

Es uno de los errores más frecuentes en UML.

Problema:

- Se utilizan asociaciones, agregaciones o composiciones de forma equivocada.
- Se representan relaciones que no existen en el dominio real.

Ejemplo: Usar composición entre Cliente y Pedido, aunque el pedido pueda mantenerse registrado aun cuando el cliente sea eliminado.

4. Uso incorrecto de herencia

La herencia debe utilizarse únicamente cuando exista una verdadera relación "es un".

Problema:

- Se crean jerarquías innecesarias.
- Se dificulta la reutilización y mantenimiento.

Ejemplo: Empleado hereda de Departamento.

Corrección: Un empleado pertenece a un departamento, no es un departamento.

5. Falta de atributos o métodos importantes

Algunas clases son diseñadas incompletas y no representan correctamente sus responsabilidades.

Problema:

- El modelo no refleja el comportamiento real del sistema.
- Genera inconsistencias durante la implementación.

Ejemplo: Una clase CuentaBancaria sin atributo saldo o sin método depositar().

6. Multiplicidades incorrectas

La multiplicidad indica cuántos objetos participan en una relación.

Problema:

- Relaciones mal interpretadas.

- Errores en la lógica del sistema.

Ejemplo: Indicar que un cliente solo puede realizar un pedido, cuando en realidad puede realizar muchos.

7. Nombres poco descriptivos

Utilizar nombres ambiguos o poco claros dificulta la comprensión del diagrama.

Problema:

- Reduce la legibilidad.
- Confunde a los desarrolladores.

Ejemplo: Clases llamadas Dato1, ClaseA o ProcesoX.

Corrección:

Usar nombres claros como:

- Cliente
- Factura
- Inventario

8. Exceso de detalle en etapas tempranas

Agregar demasiados atributos, métodos o detalles técnicos durante el análisis inicial puede complicar el modelo.

Problema:

- El diagrama se vuelve difícil de entender.
- Se pierde el enfoque del dominio del problema.

Recomendación: Comenzar con un modelo simple y refinarlo progresivamente.

[illegible]

Sección 5:

5.5.1 Software y herramientas para la creación de diagrama de clases

Actualmente existen diversas herramientas y programas que facilitan la creación de diagramas de clases UML. Estas aplicaciones permiten diseñar, organizar y documentar sistemas de software de manera visual, ayudando a los desarrolladores y analistas a representar correctamente la estructura del sistema.

1. StarUML

Es una de las herramientas más utilizadas para modelado UML profesional.

Características:

- Interfaz sencilla y moderna.
- Soporte para múltiples diagramas UML.
- Generación de código.
- Compatible con diferentes lenguajes de programación.

Ventajas:

- Fácil de usar.
- Ideal para proyectos académicos y profesionales.

2. Visual Paradigm

Herramienta completa para análisis, diseño y documentación de software.

Características:

- Creación avanzada de diagramas UML.
- Trabajo colaborativo.
- Integración con bases de datos y código.

Ventajas:

- Muy completa.
- Adecuada para proyectos grandes.

3. Lucidchart

Plataforma en línea para crear diagramas de manera rápida y colaborativa.

Características:

- Funciona desde el navegador.
- Permite trabajo en tiempo real.
- Plantillas UML prediseñadas.

Ventajas:

- Fácil acceso desde cualquier dispositivo.
- Excelente para trabajo en equipo.

4. Draw.io

También conocida como diagrams.net, es una herramienta gratuita muy popular.

Características:

- Uso gratuito.
- Compatible con Google Drive y OneDrive.
- Permite crear diagramas UML fácilmente.

Ventajas:

- Ligera y accesible.
- No requiere instalación obligatoria.

5. Microsoft Visio

Aplicación profesional para crear diagramas técnicos y empresariales.

Características:

- Plantillas UML integradas.
- Integración con herramientas Microsoft.
- Diagramas de alta calidad visual.

Ventajas:

- Interfaz profesional.

- Amplias opciones de personalización.

6. Enterprise Architect

Software orientado a proyectos empresariales y de gran escala.

Características:

- Modelado UML avanzado.
- Gestión de requisitos.
- Ingeniería inversa y generación de código.

Ventajas:

- Muy potente para proyectos complejos.
- Amplias funcionalidades de análisis.

7. PlantUML

Es una herramienta basada en texto que permite generar diagramas UML mediante código simple y legible.

Características:

- Creación de diagramas usando lenguaje textual.
- Compatible con múltiples IDE y editores.
- Generación automática de diagramas UML.
- Soporte para diagramas de clases, secuencia, casos de uso y más.

Ventajas:

- Rápido y eficiente para desarrolladores.
- Fácil integración con documentación técnica.
- Ideal para control de versiones en proyectos de software.

Bibliografía

PlantUML. <https://plantuml.com/es/>

(n.d.). Lucidchart | Diagramming Powered By Intelligence.

<https://www.lucidchart.com>

(n.d.). Visual Paradigm - AI-Powered Visual Modeling Platform. <http://visual-paradigm.com/>

(n.d.). StarUML. <https://staruml.io/>

(n.d.). Diagrams.net. <https://app.diagrams.net/>

▷ Diagrama de clases. Teoria y ejemplos. (n.d.). DiagramasUML.com.

<https://diagramasuml.com/diagrama-de-clases/>

Jacobson, I. (1992). Object-oriented Software Engineering: A Use Case Driven Approach. ACM Press.

Modelo del Dominio del Problema y Representación en UML. (n.d.).

<https://academicos.azc.uam.mx/>.

https://academicos.azc.uam.mx/jfg/diapositivas/ads/Unidad_6.pdf